



Bio-inspired model for oil production prediction: combining gene regulation-based optimization and radial basis function network

Bao Liu¹ · Yuqing Liang¹ · Zirun Zhu¹ · Lei Gao²

Received: 25 December 2023 / Accepted: 10 April 2025
© Crown 2025

Abstract

Oil production prediction plays a crucial role in the process of oilfield development, deployment, and production planning, especially when it comes to reservoir management, yet its accuracy is often unsatisfying. To address this, we proposed a bio-inspired model for oil production prediction, which combined a new genetic algorithm, known as DNA-GA, with a radial basis function (RBF) neural network. The DNA-GA was designed based on the regulation mechanism of DNA and was applied to optimize the RBF network together with the gradient descent method. The DNA-GA can effectively improve the population's diversity and prevent premature convergence, resulting in high convergence accuracy and excellent optimization ability. The tests based on Schaffer and Rosenbrock functions showed that the convergence accuracy of DNA-GA was increased by 65.38% and 67.69%, respectively, compared with the existing optimization algorithms such as the Firefly algorithm, and the convergence speed was also increased by 50%. Applied into real-world oil fields, the bio-inspired model greatly outperformed traditional neural network models in both prediction accuracy and convergence speed. Comparative analyses against the traditional RBF neural network and GA-optimized RBF neural network revealed a substantial enhancement in the prediction performance of the bio-inspired model. Specifically, the mean absolute error (MAE) was reduced by 75% and 50%, respectively. Moreover, convergence speed, as measured by average convergence iterations, was improved by 57% compared to the GA-optimized RBF neural network. These results proved that the combination of the RBF neural network and the DNA-GA algorithm could greatly improve the prediction accuracy of oil production.

Keywords Bio-inspired model · Genetic algorithm · RBF neural network · DNA · Oil production prediction

1 Introduction

The increasing demand for oil resources due to rapid economic development requires efficient oil exploitation. With rapidly declining reserves, oilfield production continuously

decreases and economic returns suffer. To maintain the stability of oil production capacity, water injection is often used in oilfield exploitation. However, water injection changes the characteristics of oil wells and affects future oil recovery. Therefore, accurate prediction of oil production under water injection is necessary to understand oil and water characteristics of oil wells, guide oilfield development, and improve production outcomes.

The oilfield injection and production system is inherently complex, nonlinear, and influenced by a number of factors with uncertainty and interdependencies. This makes it challenging to develop a precise injection-production model for predicting oilfield production. In recent decades, a variety of models have been developed to predict oilfield production, including traditional mathematical or physical models and machine learning models. Traditional models, including the regression analysis, attenuation curve method and numerical simulation method, have been widely used in oil production prediction. However, these methods present several

✉ Lei Gao
lei.gao@csiro.au

Bao Liu
liubao@upc.edu.cn

Yuqing Liang
2763769480@qq.com

Zirun Zhu
1097306866@qq.com

¹ College of Control Science and Engineering, China University of Petroleum, Qingdao 266580, China

² Commonwealth Scientific and Industrial Research Organisation (CSIRO), Waite Campus, Urrbrae, SA 5064, Australia

limitations in practical applications. For instance, Albertoni and Lake employed multiple linear regression to predict oil production and analysed reservoir injection. However, this method assumes linear relationships among variables, making it inadequate to capture the nonlinear characteristics prevalent in oil wells [1, 2]. Salazar et al. combined attenuation curves with a capacitive reactance model based on the principle of superposition. Nevertheless, this approach overlooks the geological complexities and struggled to apply linear superposition principles to the predominantly nonlinear behaviour of most reservoir data [3, 4]. Furthermore, Zhang et al. utilized numerical simulation methods for reservoir modelling and prediction. While these methods offer detailed insights into subsurface flow dynamics, their accuracy heavily relies on the construction of physical models, often resulting in limited predictive accuracy and high computational costs. In summary, traditional oil well production prediction model suffers from insufficient nonlinear modelling capabilities, poor applicability and low prediction accuracy.

To address the limitations of traditional models, machine learning model have been applied in the oilfield extraction process to predict oil production. Machine learning models overcome the shortcomings of traditional models in dealing with nonlinear problems and time series dependence through its adaptability and powerful data processing ability, such as K-means clustering [6, 7], support vector machine [8, 9], random forest [10, 11], XGBoost [11, 12] and neural networks [13, 14]. Among these machine learning methods, neural network models are the most widely used due to their superior ability of mapping nonlinear relationship and their flexibility in adjusting parameters, such as the number of hidden layers and learning rate based on specific circumstances. For example, Cheng et al. proposed a dynamic adaptive Back Propagation (BP) neural network model [15]. This model uses genetic algorithm and particle swarm optimization to optimize the parameters of BP neural network. However, the accuracy of this model prediction still needs improvement. In another study, Zhang et al. predicted the oil well production through RBF neural network and support vector machine [16]. However, the model was less efficient in the optimization process and was prone to falling into local optima.

Although intensive research has been conducted on the modeling and prediction in oil production prediction, traditional neural network models still suffer from slow convergence rates and a propensity for falling into local minima [17, 18]. To further improve the prediction accuracy of the traditional neural network, intelligent optimization algorithms were included into the neural network training process. Among them, genetic algorithm has attracted more attention of researchers because of its strong search ability and wide applicability, and it is often applied in feature selection and hyperparametric optimization of machine learning. Alfarizi et al. introduced the genetic optimization algorithm into the

artificial neural network model to effectively overcome the limitations of traditional neural networks, such as large output error and susceptibility to local optima [19]. However, the traditional genetic algorithms are also prone to be premature convergence and local optimum. To address this issue, Song et al. improved immune genetic algorithm based on greedy thought and local scheduling rules, which not only improved the learning ability and robustness of the oil production prediction model, but also enhanced its searching ability [20]. An immune cloning algorithm with multi-objective optimization was proposed, utilizing enhanced crossover and mutation operators, which outperformed other improved genetic algorithms in terms of solution quality and computational cost [21]. Despite these advancements, further improvement in accuracy is still urgently demanded. Recently, the idea inspired by human body system provides a new idea for improving the traditional optimization algorithm [22, 23], and the improvement effect of the network model of gene regulation is more obvious. For example, Zang et al. proposed a hybrid genetic algorithm called CM-DNAGA, which utilized DNA encoding and decoding methods. Based on the cloud model, the Y conditional normal cloud generator from the cloud model was employed as the crossover operator of GA, while the basic normal cloud generator was used as the mutation operator in the mutation stage. These enhancements contributed to improvements in both the quality and operational efficiency of the oil production prediction model [24]. Although this algorithm incorporates bio-inspired coding and the properties of the cloud model, it cannot fully overcome the limitations of the genetic algorithm, such as premature convergence and unreliable convergence accuracy. These studies demonstrate that the combination of the traditional optimization algorithm has made progress in improving the effectiveness of the neural network model in predicting oil production, but the prediction accuracy is still unsatisfying.

To address the drawbacks of the abovementioned improvement schemes that easily get trapped in local optima, and have insufficient prediction precision in oil production prediction, we proposed a bio-inspired optimisation algorithm to adjust neural network parameters in real-time, avoiding increased computation. Building upon traditional genetic operators, we designed a new adaptive operator by incorporating the replication, transcription, and mutation mechanisms of biological gene DNA molecules. This further led to the development of a genetic optimization algorithm based on gene regulation mechanisms, called DNA-GA.

In this study, we firstly established an oil production prediction model using the traditional RBF neural network, then we utilized the DNA-GA algorithm to enhance the gradient descent optimization method of RBF neural network. The main contributions of this paper are summarized as follows. (1) DNA-GA algorithm utilizes DNA base pairing method

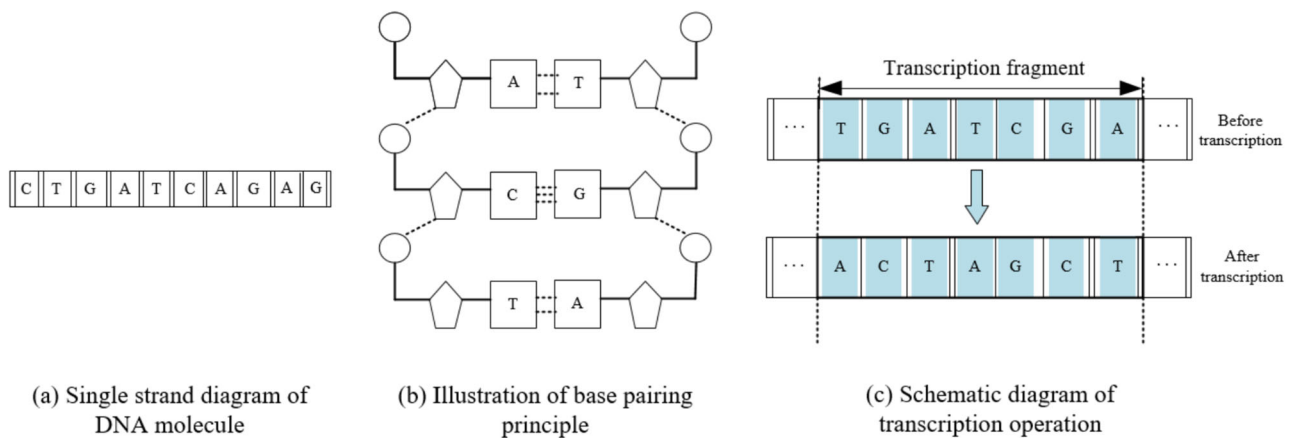


Fig. 1 Mechanisms of gene expression regulation

to encode, and enhances the selection, crossover and mutation stages of genetic algorithm. (2) In the selection stage, the combination of roulette wheel selection and sorting are applied to improve the selection operator. (3) In the crossover stage, new individuals are transcribed based on adaptive transcription factors and under the guidance of complementary base pairing principle. (4) In the mutation stage, substitution mutation and recombination mutation occur according to the specific mutation rate of DNA gene. By reducing the unstable optimization effect of traditional genetic algorithms, we showed that DNA-GA had superior performance. This further improved the prediction accuracy of the oil production prediction model, achieving satisfying accurate prediction. Our study confirmed that integrating a bio-inspired optimization algorithm could significantly improve the accuracy of oil production predictions, which provided a promising way to enhance the accuracy of oil production prediction during oil-field injection and production.

2 Gene regulation in humans

Gene regulation is a mechanism that controls gene expression within organisms, and is primarily accomplished through the replication, transcription and translation of DNA molecules. This process occurs mainly at three levels: (1) control replication, transcription and translation of genes at the DNA molecular level; (2) change metabolism at the microbial level; and (3) cellular differentiation and development at the multicellular biological level. Understanding gene regulation is crucial for understanding inheritance and evolution in living organisms.

2.1 Mechanisms of gene expression regulation

Gene expression regulation occurs at the transcriptional level, controlling the growth of organisms through the replication, transcription and translation of DNA molecules. DNA

molecule is a double-helical macromolecular compound consist of four nucleotides: adenine deoxynucleotide (A), cytosine deoxynucleotide (C), guanine deoxynucleotide (G) and thymine deoxynucleotide (T). The single chain of DNA is regarded as a string composed of four different letters: A, G, C and T [25].

The process of replication and transcription of DNA is catalyzed by polymerase, using single-stranded DNA as template and four deoxynucleotides as materials. When the activity of enzyme reaches a certain level, transcription occurs according to the base complementary pairing principle: base A corresponds to base T and base C corresponds to base G, then generates new DNA molecular chains and completes the transmission of genetic information. The process is shown in Fig. 1.

2.2 Mechanisms of gene mutation

Gene mutation in biology refers to the sudden heritable variation in DNA molecules. Changes in the composition or order of base pairs can lead to gene mutations. According to the changes in the DNA molecules bases, gene mutation can generally be divided into base replacement mutation, shift mutation, deletion mutation and insertion mutation, as illustrated in Fig. 2.

Gene mutations can occur spontaneously or be induced. induction can result in organisms producing large and diverse numbers of genetic mutations, and generate more new genetic information, then cultivate excellent varieties according to demand, which is greatly significant to biological evolution.

With the integration of mathematical tools, many theoretical models have emerged in gene regulatory networks. The expanding research results have shown great application value in the medical field. In addition, gene regulatory network, as a cross-disciplinary field encompassing mathematics, informatics, computer sciences and biology, has much reference value in system design [26] and improvement

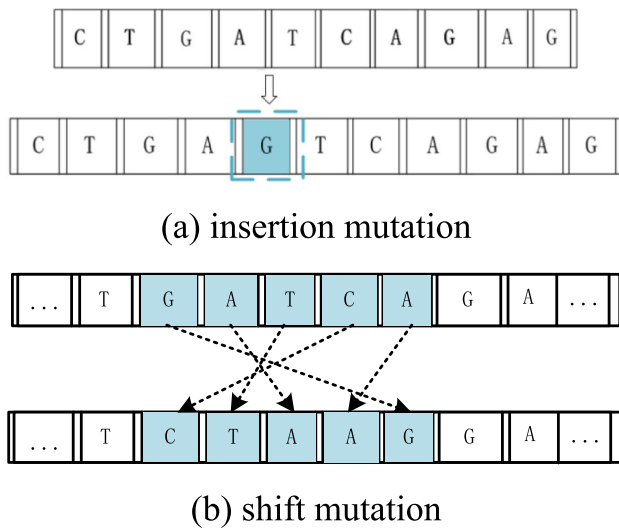


Fig. 2 Gene mutation process

of group optimization algorithms [27]. The new optimization algorithms based on gene regulation network mechanisms are crucial for improving the prediction accuracy of oil production.

3 Optimizing RBF genetic network using the DNA-GA algorithm

3.1 DNA-GA

Genetic algorithm (GA) is an adaptive global optimization algorithm that mimics the genetic and evolutionary processes of organisms in nature [28]. It finds the optimal solution to the target function by encoding it as chromosomes through selection, crossover and mutation. Genetic algorithm has the characteristics of wide application range and strong robustness, which can find the global optimal solution of the optimization problem and effectively prevent the neural network from falling into the local optimal problem in the training process. However, traditional coding methods can lead to premature convergence, where the optimization ability of GA weakens once the error reaches a certain value, thereby not fully demonstrating the role of gene regulation. In recent years, efforts have been made to improve the traditional GA by enhancing the crossover and variation process [29, 30], yet the issue of premature convergence persists. To address the limitation of GA, this paper proposes the DNA-GA based on gene regulation and upgrades the traditional GA by enhancing the individual coding mode, replication, crossover and mutation process. As a result, the optimization accuracy and speed of DNA-GA are improved.

Table 1 shows the corresponding relationships between the modules of DNA-GA and the mechanisms of human

Table 1 Corresponding relationships between DNA-GA modules and mechanisms in the gene regulation in humans

DNA-GA		Gene Regulation in Human	
Module	Role	Mechanism	Role
Individual coding	Guide individuals to encode in quaternary coding method	Principle of complementary base pairing	Guiding the synthesis of DNA double-stranded by complementary pairing of A-T and C-G
Replication	Guiding the improved selection operator to copy the parent individual	Principle of survival of the fittest	Individuals or genes that can adapt to nature are selected to survive
Transcription	Guiding the transcription of adaptive transcription factor to produce new individuals	Transcription	Guiding genetic information to flow from DNA to RNA under the action of RNA polymerase
Mutation	Guiding the mutation of adaptive mutation factor to generate new individuals	Mutation	Causing sudden and heritable variation on DNA molecules

gene regulation. Inspired by the principle of base complementary pairing in DNA, DNA-GA adopted a quaternary coding method to encode individuals. Secondly, inspired by the principle of survival of the fittest in nature, the selection operator was improved by combining roulette and sorting to select parent individuals for replication. Then, inspired by DNA transcription, adaptive transcription factors were utilized to guide the transcription process and produce new individuals. Finally, inspired by the displacement mutation and recombination mutation in the process of gene mutation, the adaptive mutation factor was improved to enhance the ability to escape local optima.

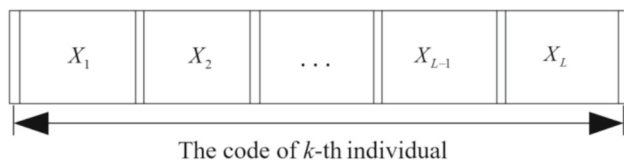


Fig. 3 Quaternary base codes for an individual population

3.1.1 Core principles

(1) Individual coding

The DNA-GA algorithm uses a coding method based on bases in the DNA chain. A single DNA molecule can be expressed as a set of four letters: {A,T,C,G} (Fig. 1). As a result, chromosomes are encoded in a quaternary coding method. In the method, 0 represents base A, 1 represents base C, 2 represents base G, and 3 represents base T. Initial individuals are represented by randomly generated quaternary number sequences. For example, $X = 203,103,210,322,031$ represents an individual of length 15. If the population of individuals reaches N , the structure of the k -th ($1 \leq k \leq N$) individual is shown in Fig. 3. L represents the length of the individual code, $X_1, X_2, \dots, X_{L-1}, X_L$ represent the parameters value, and the value of X_i ($1 \leq i \leq L$) is an integer between zero and three.

The quaternary base coding method developed in this paper is inspired on the coding mode of DNA molecules in gene regulation. The individuals' coding mode ignores the order and number of bases. The replication, transcription and gene mutation of DNA in base expression be incorporated into the selection, crossover and mutation of the DNA-GA, and the information is processed according to the principle of base complementary pairing.

(2) Replication

The selective replication imitates the process of natural survival by retaining individuals with large process fitness values. Traditional genetic algorithms use the roulette method to randomly select individuals to replicate, preserving the diversity of the next generation. However, when fitness differences between individuals become small in later stages of evolution, the convergence rate slows down. In this study, the selection operator is improved by combining the roulette method with sorting. The roulette method is still used in the early stage of evolution to maintain the population diversity; while in the middle and later stages, where fitness differences between individuals are small, to better reflect the principle of natural survival during biological evolution, selection by sorting is adopted. Individuals are sorted by fitness values in descending order, with only those with higher fitness being

retained. The improved selection probability of the k -th individual $p(x_k)$ is calculated as follows:

$$\begin{cases} p(x_k) = \frac{q}{1-(1-q)^N} \times [(1-q)^{k-1} + (1-q)^{N-k+1}] & (k \leq \frac{N}{2}) \\ p(x_k) = 0, & (k > \frac{N}{2}) \\ q = fit_{avg} / \sum_{i=1}^{N-\frac{N}{2}+1} fit(x_i) \end{cases} \quad (1)$$

where, N is the number of individuals in the population, k is the serial number of individuals sorted by fitness, q is the average fitness selection probability, fit_{avg} is the average of individuals' fitness in the population, and i is the serial number of individuals before fitness ordering.

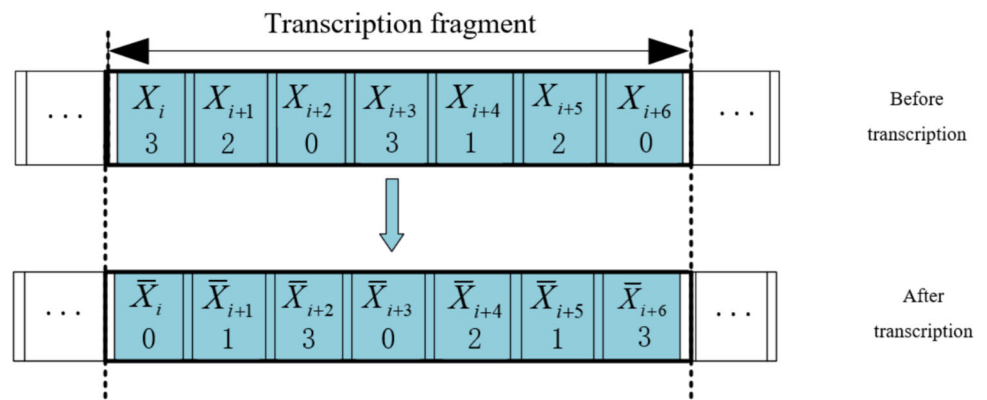
The probability of individual selection is closely related to the fitness value. Firstly, we rank the fitness values of individuals from small to large. When k is less than $\frac{N}{2}$, we calculate the probability that the current individual is selected according to the average fitness selection probability q , as shown in formula (1). When k is greater than $\frac{N}{2}$, we set the probability of an individual being selected to 0, that is, not being selected.

(3) Transcription

In the traditional genetic algorithm, the crossover combines the information from the parents according to a certain crossover rate to form new superior individuals. The paternal individuals in population A generally adopt linear adaptive crossover rate. However, the process will reduce the diversity of populations and leads to inbreeding, which increases the risk of premature convergence and getting stuck into local optimum.

To address the above challenge, inspired by the transcription process of DNA, we propose to improve the adaptive crossover process of traditional genetic algorithms. Individuals are transcribed based on transcription factors, and different gene fragments from the parental individuals are selected when the transcription factor reaches different activity values. Then new individuals are generated after transcription according to the principle of base complementary pairing, preserving the population diversity and effectively avoiding premature convergence and local optimum. Transcription determines the origin and length of the operations by using transcription factors. If the variant factor is p_c , the length of transcription is generally $a = L \cdot p_c$. Transcribed fragments can be expressed as: $[X_{L-a}, X_{L(1-p_c)}, \dots, X_L]$, where L is the length of the individual encoding. The transcription process is shown in Fig. 4.

If the starting point is x_i , the individual before transcription can be expressed as: $X = [X_{i+1}, X_{i+2}, \dots, X_{i+n}]$; And the individual after transcription can be expressed as: $\bar{X} = f(x) = [\bar{X}_{i+1}, \bar{X}_{i+2}, \dots, \bar{X}_{i+n}]$; where, $\bar{x}_i$ indicates

Fig. 4 Schematic diagram of transcription operation

the quaternary reverse code of x_i , the length of transcription is $n = L \cdot p_c$ and integrated down.

In biology, the transcription process of DNA is facilitated by the actions of transcription factors, which determines the origin and gene number. The adaptive transcription factor named p_c based on the logistic function are defined according to the specific non-linearity of human transcription factors:

$$p_c = \begin{cases} p_0 \times \exp\left(\frac{fit_{\max} - fit}{fit_{\max} - fit_{\min}}\right), & fit > fit_{avg} \\ p_0, & fit \leq fit_{avg} \end{cases} \quad (2)$$

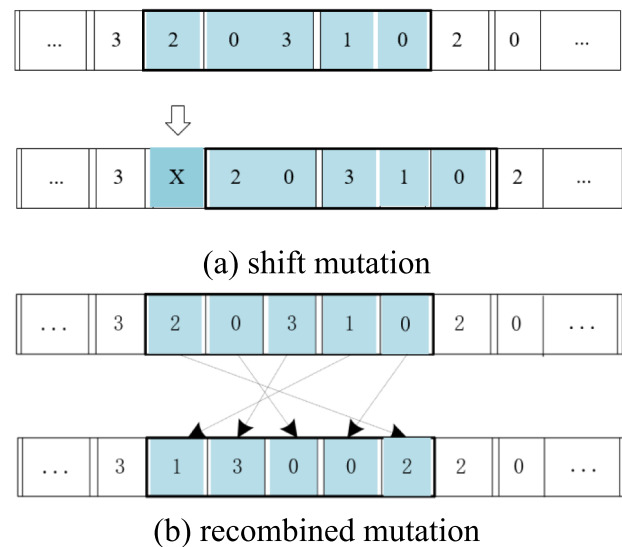
where, p_0 is the initial transcription factor, fit is the individual's fitness before transcription, $fit_{\max}$ is the maximum individual's fitness in the population, $fit_{\min}$ is the minimum individual's fitness in the population, and fit_{avg} is the mean of individual's fitness in the population.

Transcription factors are associated to individuals' fitness. Initially, the transcription factor p_0 is set to 0.6. During the iterative process, if fit exceeds fit_{avg} , the transcription factor p_c is updated using an exponential function of formula (2), that is, the higher the fitness, the smaller the factor, conversely, if fit is lower than fit_{avg} , the initial transcription factor remains unchanged.

(4) Mutation

In the mutation of a genetic algorithm, the parent individual undergoes code changes to form a new offspring individual with a certain probability of mutation. mutation reflects genetic algorithms' local optimization. However, traditional genetic algorithms are prone to premature convergence. When the error of the target function reaches to a certain level, the optimization efficiency decreases, leading to a risk of getting stuck in local optimum.

This work further improves the mutation of traditional genetic algorithms. The improved mutation includes two types: shift mutation and recombined mutation. The shift mutation is the mutation caused by gene insertion. As shown in Fig. 5a, X is randomly inserted at the start position of the

**Fig. 5** Variation operation diagram (Blue bits represent gene fragments)

mutation and the subsequent encoding is successively shifted backward to form new individuals. On the other hand, recombined mutation follows the principle of the gene random shift mutation, disrupting the coding order of the mutational fragments and then randomly combines them to form new individuals, as shown in Fig. 5b. Mutation determines the transcription location and length by the mutation factor, if the variant factor is p_c , the length of mutation is generally $b = L \cdot p_c$ and integrated down. Mutational fragments can be expressed as: $[X_{L-b}, \dots, X_L]$, where L is the length of the individual encoding, as shown in Fig. 5.

As depicted in Fig. 5, there are two modes of variation in this work, and they can be randomly selected to manipulate an individual in each iteration. To increase the randomness of the variation selection, these modes are determined according to the interval of the random value m between 0 and 1, which is generated randomly and the middle value of 0.5 in the interval is selected as the partition point: If $(0 < m \leq 0.5)$, using

the shift mutation; If $(0.5 < m \leq 1)$, using the recombined mutation.

In biological evolution, the occurrence of gene mutation and the fitness of the individual in the environment are closely related. Individuals with higher fitness have the stronger stability and a lower probability of mutation. To reflect this relationship, an adaptive mutation factor q_c based on the Sigmoid function was defined as:

$$q_c = q_0 \times \exp\left(\frac{fit_{\max} - fit}{fit_{\max}}\right) + \frac{1}{1 + q_0} \times \exp^{-\frac{\Delta fit_{\max}}{\Delta S}} \quad (3)$$

where, q_0 is the initial mutation factor, fit is the fitness before mutation, $fit_{\max}$ is the maximum individual fitness in the population, $\Delta fit_{\max}$ is the amount of changes in the maximum individual fitness in the population, ΔS is the generation number separated between the current population and the population whose latest maximum fitness has been changed, and $\Delta fit_{\max} / \Delta S \geq 0$.

As indicated by formula (3), the individual mutation factor is related to the change rate of the individual fitness and the optimal individual fitness of the population. When the fitness and change rate are higher, the mutation factor is smaller, preventing the superior population from being destroyed. On the contrary, a larger mutation factor can preserve the population diversity and enhance the algorithm's capability of local optimization.

The transcription process of the algorithm is in motion according to the gene DNA expression. Specific fragments are selected to transcribe individuals and the transcription process is mapped as 0 corresponds to 3 and 1 corresponds to 2. The variation process is designed based on the gene mutation mechanism, and specific fragments are selected for individual variation operation. The variation mode is randomly selected, enabling the code to be repaired.

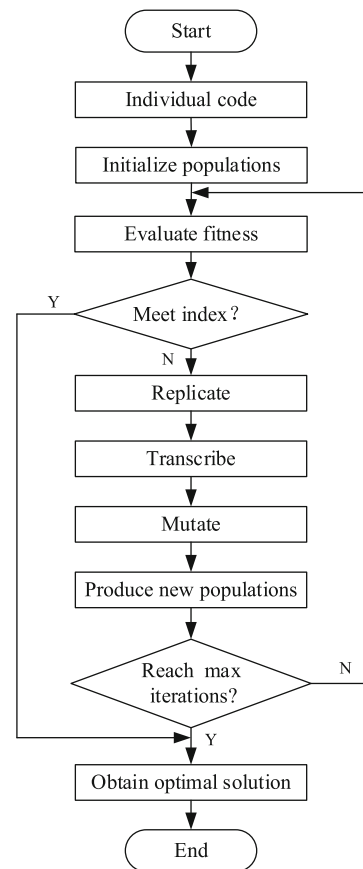


Fig. 6 Structure of DNA-GA

3.1.2 Optimization process of DNA-GA

In DNA-GA, the individual is created by the quadrilateral coding, then it uses the improved genetic operators and adjusts themselves adaptively with the fitness function. The flowchart of the above process is shown in Fig. 6.

The specific process is as follows:

Step 1: Establish the objective function optimization model according to the decision variables and constraints.

Step 2: Encode the individuals according to the quaternary coding method to generate the initial population.

Step 3: Calculate the fitness of each individual to determine whether the conditions for error iteration are met. If so, go to Step 8, otherwise go to Step 4.

Step 4: According to the improved selection operator of formula (1), the parent individual with high fitness value is copied.

Step 5: Identify the adaptive transcription factor of formula (2), and transcribe the parents according to the principle of complementary base pairing.

Step 6: Determine the adaptive mutation factor according to formula (3), and generate a new population through mutation.

Step 7: Determine whether the maximum number of iterations has been reached. If so, go to Step 8, otherwise go to Step 3.

Step 8: Obtain the optimal solution.

The optimization process of DNA-GA algorithm:

3.1.3 Complexity analysis of DNA-GA

In the time complexity analysis of the DNA-GA algorithm, we utilized the big O notation here. The big O notation usually ignores constants, low orders, and coefficients, describing the growth trend of the algorithm as the data size increases, and does not represent the actual execution time. Therefore, we calculate the time complexity for each module, then perform addition operations, and finally only retain the highest order of magnitude terms.

This article sets the maximum number of iterations as G , the population size as N , the variable spatial dimension as D , and the objective function as M . The time complexity of initializing the population is $O(N \cdot D)$, the time complexity of the fitness function is $O(N \cdot M)$, and two sorting operations are used during the algorithm iteration process. The time complexity is $O(2N \cdot (\log N) \cdot M)$, the time complexity of the replication process is $O(N^2 \cdot M)$, the time complexity of the transcription process is $O(N \cdot M)$, and the time complexity of the mutation process is $O(N \cdot M)$. In summary, the total time complexity $O(N \cdot D + N \cdot M + 2N \cdot (\log N) \cdot M + N^2 \cdot M$

$+ N \cdot M + N \cdot M + N \cdot M) = O(N^2 \cdot M)$. Compared with other intelligent optimization algorithms, the computational cost has increased.

3.2 RBF neural network optimized by DNA-GA

3.2.1 RBF principle

Neural networks, including the RBF type, are capable of performing non-linear operations by simulating mechanisms inspired from the human brain, with strong self-learning and adaptive ability, so they are frequently employed in predictive models.

The RBF neural network is a typical three layers feedforward network with input layer, hidden layer and output layer [31]. The first layer is the input layer, consisting of signal source nodes that solely transmit data without transforming input information. The second layer is the hidden layer, with a variable number of nodes based on specific requirements, featuring neurons with Gaussian function kernel. This layer

spatially maps input information. The third layer is the output layer, responsive to the input mode. Neurons in the output layer utilize a linear function, linearly weighting the output of neurons in the hidden layer to produce the overall neural network output. The structure of RBF neural network is shown in Fig. 7.

In Fig. 7, i is the number of input nodes, j is the number of hidden nodes, m is the number of output nodes, $X = [X_1, X_2, X_3, \dots, X_i]$ is the input data, $H = [H_1, H_2, H_3, \dots, H_j]$ is the hidden data, $Y = [Y_1, Y_2, Y_3, \dots, Y_m]$ is the output data.

3.2.2 RBF optimized by DNA-GA

When predicting oilfield production, traditional statistical methods are not sufficiently accurate due to the nonlinearity and uncertainty in the prediction problem. The RBF neural network model is promising to address the challenge [32]. However, it may face low convergence rate and local optimum when dealing with complex problems such as nonlinearity and lag nature. To improve the accuracy of the prediction model, this paper combines the DNA-GA algorithm and the momentum gradient training to optimize the parameters of RBF neural network. The structure of RBF neural network optimized by DNA-GA is shown in Fig. 8.

In Fig. 8, c_{ij} is the coordinate vector of the Gaussian function's central point in the hidden layer, the same dimension as the input data, $\|x - c_{ij}\|^2$ is defined as the Euclidean distance between the input layer and the central vector, b_j is the width of the Gaussian function in hidden layer, h is output of the hidden layer and w_j is the weights of the hidden layer to the output layer. $y = h_j w_j$ represents the actual output, E is the error indicator function, and the hidden layer is activated

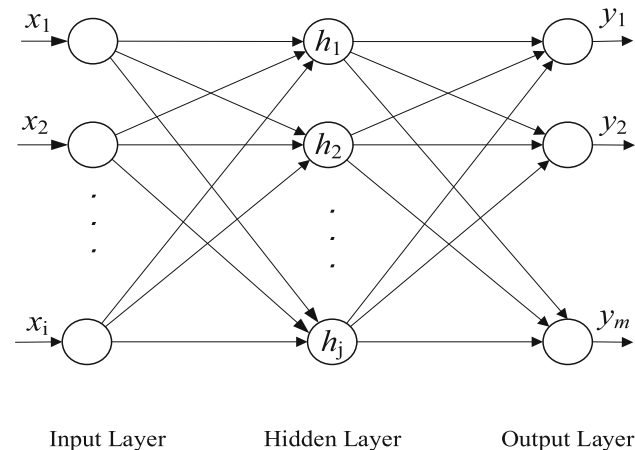


Fig. 7 Structure of RBF neural network

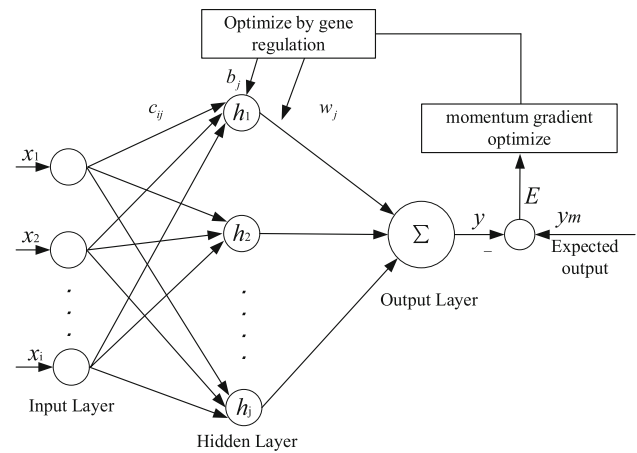


Fig. 8 Structure of RBF neural network optimized by gene regulation

by Gaussian function:

$$h_j = \exp\left(-\frac{\|x - c_j\|^2}{2b_j^2}\right) \quad (4)$$

The momentum gradient training process is as follows:

$$w_j(k) = w_j(k-1) + \Delta w_j(k) + \alpha[w_j(k-1) - w_j(k-2)] \quad (5)$$

$$\Delta w_j(k) = -\eta \frac{\partial E_k}{\partial w_j} = \eta e h_j \quad (6)$$

$$b_j(k) = b_j(k-1) + \Delta b_j(k) + \alpha[b_j(k-1) - b_j(k-2)] \quad (7)$$

$$\Delta b_j(k) = -\eta \frac{\partial E_k}{\partial b_j} = \eta e w_j h_j \frac{\|x - c_j\|^2}{b_j^3} \quad (8)$$

$$c_{ji}(k) = c_{ji}(k-1) + \Delta c_{ji}(k) + \alpha[c_{ji}(k-1) - c_{ji}(k-2)] \quad (9)$$

$$\Delta c_{ji}(k) = -\eta \frac{\partial E_k}{\partial c_{ji}} = \eta e w_j h_j \frac{x_j - c_{ji}}{b_j^2} \quad (10)$$

In the above formulas, $\eta \in (0, 1)$ is learning efficiency and $\alpha \in (0, 1)$ is momentum factor.

We used the error absolute value accumulation index as the minimum objective function of parameter selection:

$$J = \sum_{i=1}^M |y(i) - y_m(i)| \quad (11)$$

In this approach, we used the error indicator function of the RBF neural network as the fitness evaluation function.

The training process of the RBF neural network optimized by DNA-GA is shown in Fig. 9.

The steps of training the RBF neural network with gene regulatory genetic algorithm combined with momentum gradient training are applied as follows:

Input: n individuals consist of parameters of RBF neural network: length of Gaussian function, b_j ; Center vector, c_{ij} ; Weight from hidden layers to the output layer, w_j ; the individual code X ; i.e., $X = (X_1, X_2, X_3, \dots, X_L)$, maximum epochs p .

Output: Optimum fitness value y_x and the corresponding individual x_o .

Step 1: Apply the gradient optimization training to optimize the neural network's parameters preliminarily. Use Eqs. (5) and (6) to calculate the w_j , Use Eqs. (7) and (8) to calculate the b_j , and use Eq. (9) (10) to calculate the c_{ij} .

Step 2: Establish the objective function F according to the m various constraints; i.e.

$F = (f_1, f_2, f_3, \dots, f_m)$; After encoding, initialize populations $x = (x_1, x_2, x_3, \dots, x_n)$.

Step 3: Evaluate individuals' fitness fit using Eq. (11) as the minimum objective function to select parameters.

Step 4: Determine the transcription factor p_c and the mutation factor q_c according to the fitness value, then through replicate, transcription and mutation to produce new populations.

Step 5: Judge whether the error index of the new population is satisfied. If so, go to **step 6**; otherwise, continue iteration. If it arrives the maximum epochs but not find the optimal solution, skip the loop.

Step 6: When the iterations stop, the individual with the optimal fitness is selected as the optimum.

Step 7: Decode the output sequence $X_o = (X_{o1}, X_{o2}, X_{o3}, \dots, X_{oL})$, obtain the optimum parameters, b_j , c_{ij} and w_j .

Step 8: End

3.3 Model performance evaluation and stability analysis

The mean absolute error (MAE), the mean relative error (MRE), mean square error (MSE) were used to compare the prediction results of the models in this work. The MAE reflects the error between the predicted value and the output value, the MRE reflects the deviation from the predicted value to the real value, and the MSE reflects the degree of difference between the predicted value and the real value.

$$MAE = \frac{1}{M} \sum_{i=1}^M |y(i) - y_m(i)| \quad (12)$$

$$MRE = \sum_{i=1}^M \frac{|y(i) - y_m(i)|}{y_m(i)} \times 100\% \quad (13)$$

$$MSE = \frac{1}{M} \sum_{i=1}^M (y(i) - y_m(i))^2 \quad (14)$$

where, M is the total number of oil production samples used for testing; $y_m(i)$ represents the true oil production of the i -th sample; and $y(i)$ represents the predicted value of the i -th sample.

The RBF neural network prediction model optimized by DNA-GA needs to guarantee the stability of the model, which also needs to be verified.

Theorem: The neural network predictive model is stable when $0 < \eta < 2 / \|h_j\|^2$.

Prove: Construct the Lyapunov function shown as follows:

$$L(k) = \frac{1}{2} [e_i(k)]^2 \quad (15)$$

Then:

$$\begin{aligned} \Delta L(k) &= L(k+1) - L(k) \\ &= \frac{1}{2} [e_i(k+1)]^2 - \frac{1}{2} [e_i(k)]^2 \\ &= \frac{1}{2} [e_i(k+1) - e_i(k)][e_i(k+1) + e_i(k)] \\ &= \Delta e_i(k) \left[e_i(k) + \frac{1}{2} \Delta e_i(k) \right] \end{aligned} \quad (16)$$

From formula (7):

$$w_j(k) = w_j(k-1) + \Delta w_j(k) \quad (17)$$

In this formula, w is the weight matrix of the network.

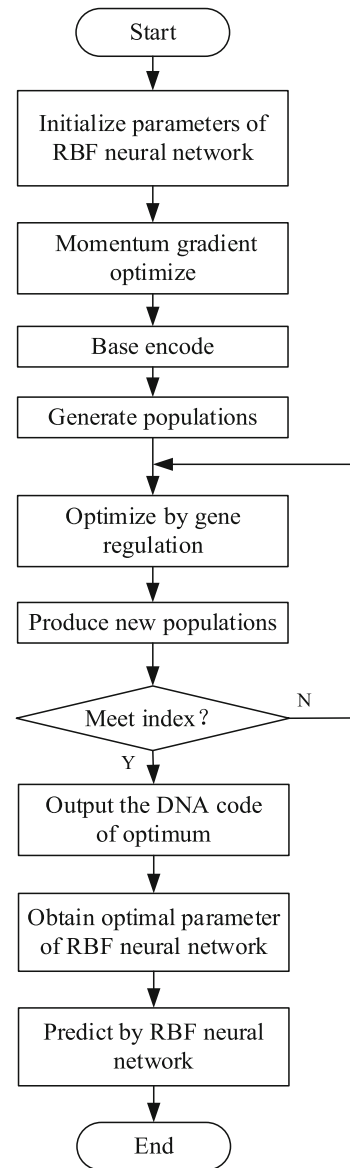


Fig. 9 Training process of RBF neural network optimized by DNA-GA

Therefore:

$$\begin{aligned} \Delta w_j(k) &= w_j(k) - \Delta w_j(k-1) \\ &= -\eta \frac{\partial E_k}{\partial \Delta w_j} = \eta e h_j \end{aligned} \quad (18)$$

It has been known:

$$\begin{aligned} \Delta e_i(k) &= \left[\frac{\partial e_i(k)}{\partial w} \right] \Delta w = -[h_j]^T \Delta w \\ &= -[h_j]^T \eta e_i(k) h_j \end{aligned} \quad (19)$$

Take formula (19) into formula (16):

$$\begin{aligned}
 \Delta L(k) &= \Delta e_i(k) \left[e_i(k) + \frac{1}{2} \Delta e_i(k) \right] \\
 &= -[h_j]^T \eta e_i(k) h_j \left[e_i(k) + \frac{1}{2} (-[h_j]^T \eta e_i h_j) \right] \\
 &= -[h_j]^T \eta e_i(k) h_j e_i(k) \left[1 - \frac{1}{2} [h_j]^T \eta h_j \right] \\
 &= -\eta e_i(k)^2 \times \|h_j\|^2 \times \left[1 - \frac{1}{2} \eta \|h_j\|^2 \right] \quad (20)
 \end{aligned}$$

where $e_i(k)^2 > 0$, $h_j \neq 0$, $\|h_j\|^2 > 0$, so it is necessary to verify $1 - \frac{1}{2} [h_j]^T \eta h_j > 0$ to ensure $\Delta L(k) < 0$. Therefore, when $0 < \eta < 2/\|h_j\|^2$, the neural network prediction model is asymptotically stable.

4 Experiment setup and results

4.1 Test functions and performance of optimization algorithms

To validate the optimization capability of DNA-GA, the Schaffer function [Formula (21)] and the Rosenbrock function [Formula (22)] were used as the test functions, based on which the performance of DNA-GA was compared with that of three existing optimization algorithms: fire-fly algorithm (FA), artificial fish swarm algorithm (AFSA), and standard genetic algorithm (GA).

The two test functions are represented as:

$$\begin{cases} f(x_1, x_2) = 0.5 - \frac{\sin^2 \sqrt{x_1^2 + x_2^2} - 0.5}{(1 + 0.001(x_1^2 + x_2^2))} \\ -5 \leq x_i \leq 5 (i = 1, 2) \end{cases} \quad (21)$$

$$\begin{cases} f(x) = \sum_{i=1}^{n-1} [100(x_{i+1} - x_i^2)^2 + (x_i - 1)^2] \\ -1 \leq x_i \leq 2, 1 \leq i \leq n \end{cases} \quad (22)$$

The two test functions have numerous local solutions in close proximity to the global solution, making it essential to avoid being trapped in local optima while searching for the global solution.

As for the comparison algorithm, the firefly algorithm and the artificial fish swarm algorithm, like the genetic algorithm, belong to natural heuristic search algorithms, which are characterized by simple principles, easy implementation, high search efficiency for complex problems, and strong robustness. They are widely used in areas such as function optimization, machine learning, and data mining. Next are the descriptions of the three existing optimization algorithms:

- (1) The Fire-fly algorithm [31] simulates the natural phenomenon of fireflies congregating at night to optimize the search. It achieves this by using the better individual to attract other individuals and eventually locking onto the optimal solution. In this algorithm, the individual firefly is equivalent to the point in the search space, and the attraction and movement process of the individual firefly are equivalent to the search and optimization process, while the position of the individual corresponds to the objective function of the problem being solved.
- (2) Artificial fish swarm algorithm [34] is a population-based optimizer that constructs artificial fish to imitate the feeding, flocking, and chasing behavior of fish shoals. The environment where an artificial fish lives is primarily the solution space, as well as the states of other artificial fish. An artificial fish's next behavior depends on its current state and its local environmental state, which includes the quality of current solutions and the states of nearby companions. An artificial fish can influence the environment via its own activities and its companions' activities.
- (3) Genetic algorithm [35] is a random search algorithm draws on the principle that survival of the fittest. It starts with an initial set of random solutions called the population, with each individual in the population is coded as a chromosome that represents a solution to the problem at hand. The chromosomes evolve through successive iterations called generations. During each generation, the chromosomes are evaluated using some measures of fitness value. The offspring are formed by either merging two chromosomes from current generation using a crossover operator or modifying a chromosome using a mutation operator. As the number of iterations increases, fitter chromosomes have higher probabilities of being selected according to the fitness function values. After several generations, the algorithm converges to the best chromosome, which represents the optimum solution.

The above optimisation algorithms were implemented in Matlab. The standard GA were set with a crossover probability p of 0.7 and a mutation probability q of 0.1 based on [36]; The parameters of the AFSA were set as follows: visual field of the swarm V was 1, length of a step S was 0.1, congestion factor D was 0.618 and repeat T was 100 according to [37]. The parameters of the FA were set as: fluorescence play factor H was 0.4, fitness ratio M was 0.6, neighbourhood change rate B was 0.08, neighbourhood threshold T was 5, length of a step S was 0.03, fluorescence concentration L was 0.5, and sensing radius R was 5 based on [38]. Finally, the DNA-GA was set with a transcription factor parameter p_0 of 0.7 and an initial variation factor q_0 of 0.1.

In all experiments, the population size was set to 40, and the maximal number of iterations was 100 generations. n in

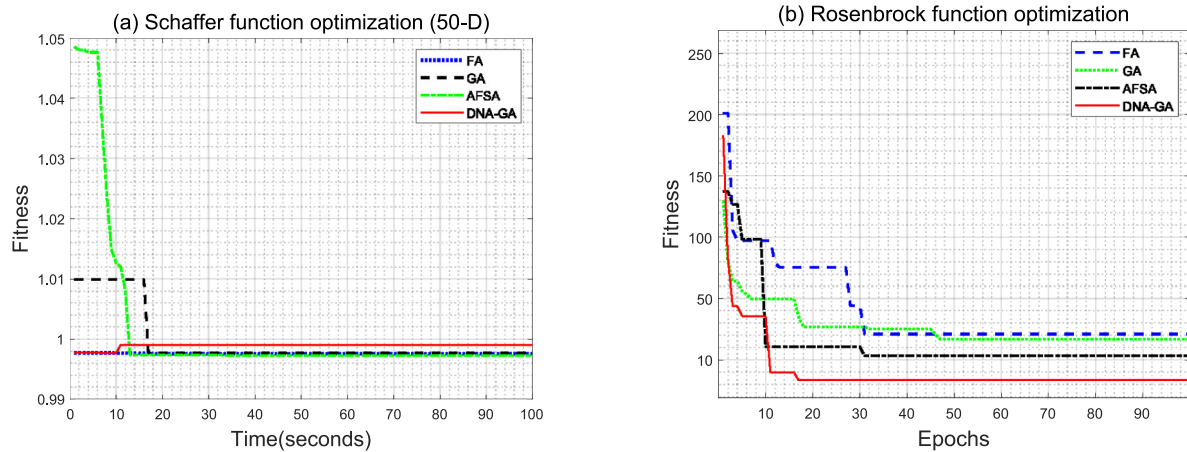


Fig. 10 Optimization curves of the four approaches for the Schaffer and Rosenbrock functions

the Rosenbrock function was set as 50, and 30 comparison experiments were performed for each algorithm. The optimization curves for the Schaffer and Rosenbrock functions are shown in Fig. 9. In the Fig. 10a, both of FA and GA took longer to converge at the optimal value, while DNA-GA had the shortest convergence time and the closest final convergent value to the actual optimal value among all algorithms. In Fig. 10b, the DNA-GA had the fastest convergence rate and its final convergent value was the minimum, which was also closer to the actual optimal value than the other three algorithms.

There are many local solutions near the global optimum of a test function, and the optimum of the Rosenbrock function lies in a long and narrow parabolic valley so that it is difficult to find it. Five performance metrics were used to compare the optimization capabilities of these algorithms, as shown in Table 2.

In this study, the DNA-GA was applied to optimize the radial basis function (RBF) neural network to enhance its convergence speed and accuracy. The Schaffer and Rosenbrock functions were used to evaluate the performance of the DNA-GA and compared with the traditional GA, AFSA, and ant FA. The results showed that DNA-GA has a significant advantage in reducing the maximum number of iterations required to search for optimal values (Table 1). In the Schaffer function experiment, the average number of iterations for DNA-GA was 32, which is a 57.33% improvement over traditional GA, 60.49% over AFSA, and 42.86% over FA. Similarly, in the Rosenbrock function experiment, the average number of iterations for DNA-GA was 36, which is a 56.62% improvement over traditional GA, 59.55% over AFSA, and 47.83% over FA. The error between the found optimum and real optimum value was used to evaluate the accuracy of the algorithms. In the Schaffer function experiment, the global maximum is 1, while DNA-GA's optimum was 0.99901 with an error of 9.9×10^{-4} , which is a 0.243%,

0.207%, and 0.189% improvement over traditional GA, AFSA, and FA, respectively. In the Rosenbrock function experiment, the global minimum is 0, while DNA-GA's optimum was 0.63 with an error of 81.0%, 67.7%, and 82.3% improvement over traditional GA, AFSA, and FA, respectively. In terms of the running time of the algorithm, the optimization time of DNA-GA was obviously longer than that of FA and ASFA, and slightly longer than that of GA. As a result, DNA-GA has an obvious superiority in convergence rate and precision but slightly insufficient in optimization time compared to other algorithms.

4.2 A bio-inspired model and its performance for oil production prediction

4.2.1 Applied background and parameter setting

This study aimed to predict oil production using a neural network nonlinear modelling method. The water injection and oil production data from over 2,000 oil and water wells in a domestic oil field were used for this purpose. The analysis on the data up to 24 months from January 2015 to December 2016 suggested that the oil production and the water injection sleeve pressure are closely related, and the injection model has hysteresis; therefore, the neural network nonlinear modelling method was adopted.

In this study, we considered the widely applicable Back-propagation (BP) and Radial Basis Function (RBF) neural networks to predict oil production. BP neural network, a type of feedforward network, is trained using a backpropagation algorithm, which propagates errors backward through the layers and updates weights via a gradient descent algorithm. BP network excels at solving nonlinear problems but can easily get stuck in local optima and requires extensive training time. RBF neural network, a radial basis function network,

Table 2 The optimization performance of the four algorithms

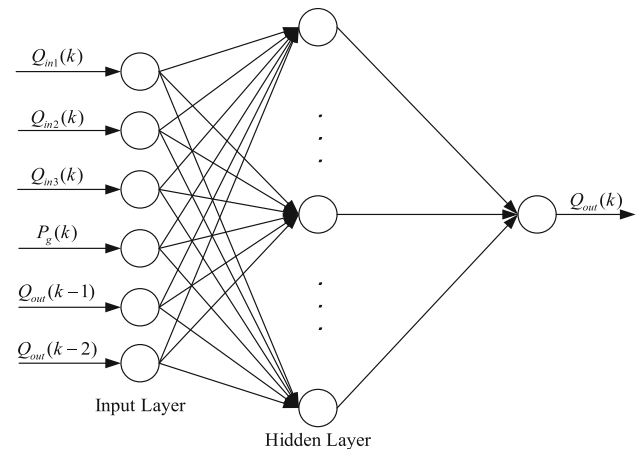
Test functions	Algorithms	Performance index				
		Minimal iterations	Maximal iterations	Average iterations	Average optimum	Running time
Schaffer	GA	9	100	75	0.99659	0.83820
	AFSA	16	92	81	0.99695	0.77540
	FA	6	76	56	0.99713	0.51720
	DNA-GA	12	56	32	0.99901	1.08280
Rosenbrock	FA	11	95	69	3.56e + 1	0.64840
	GA	15	100	83	3.32e + 1	1.12220
	AFSA	16	89	89	1.95e + 1	0.54789
	DNA-GA	13	65	36	0.63e + 1	1.20159

establishes optimal centre and radii for the radial basis functions and determines weights using the least squares method. RBF network boasts rapid training speed and high accuracy, yet it is sensitive to data noise and may encounter overfitting issues. We feed oil well data into both BP and RBF neural networks for prediction, as illustrated in Fig. 13. The RBF network demonstrated notably superior predictive performance compared to the BP network, leading us to employ the DNA-GA algorithm to enhance the RBF oil production prediction model.

The RBF neural network was optimized using DNA-GA by compiling 1,100 sets of input and output data, filtering and normalizing them using wavelet noise reduction, and then taking 1,000 data as training samples and the remaining 100 as test samples. In this work, we focus on the prediction of oil production based on four important factors/features: water injection quantity of the original well $Q_{in1}(k)$, water injection of the primary infill well $Q_{in2}(k)$, water injection of the secondary infill well $Q_{in3}(k)$ and sleeve pressure $P_g(k)$. The oil production $Q_{out}(k)$ was regarded as the output of the neural network. We use a three-layer RBF neural network model with six inputs, including the four selected features and the feedback states $Q_{out}(k-1)$ and $Q_{out}(k-2)$. Figure 11 illustrates the structure of our oil production prediction model based on the RBF neural network.

Specifically, the RBF neural network model consists of an input layer with 6 neurons, a hidden layer with seven neurons and an output layer with one neuron. We optimized the RBF neural networks using the traditional GA and the DNA-GA, and their parameter settings are shown in Table 3.

In Table 3, η is the learning rate of RBF, α is the momentum factor, E is ideal error, MG is maximum iteration. In the algorithm, N is populations' sizes, G is maximum iteration, p and q are crossover probability and mutation probability of GA, p_0 and q_0 are transcription factor parameter and initial variation factor of DNA-GA.

**Fig. 11** Structure of oil production prediction model based on the RBF neural network

4.2.2 Oil production forecast results and model comparison

By setting the above parameters, we show the error curves of GA and DNA-GA under the same number of iterations, and the experimental effects are shown in Fig. 12. It is evident from this Fig. that the DNA-GA outperform the GA in terms of both accuracy and speed of population optimization. It successfully avoids the premature convergence.

In order to show the change of fitness function in the process of algorithm optimization more intuitively, the change of individual fitness in an iteration is shown in Table 4.

To demonstrate the advantages of the DNA-GA, we constructed an oil wells injection model using the classical backpropagation (BP) neural network, conventional RBF neural network, RBF neural network optimized by GA (GA-RBF) and RBF neural network optimized by DNA-GA (DNA-GA-RBF) to predict the oil production, respectively. The experimental performance of these models on the test dataset is shown in Fig. 13.

Table 3 Parameter setting of the traditional GA, the DNA-GA, and RBF

GA				DNA-GA				RBF			
p	q	N	G	p_0	q_0	N	G	η	α	E	MG
0.6	0.05	40	100	0.6	0.05	40	100	0.10	0.85	0.001	500

Fig. 12 Iterative errors of objective function optimization by the traditional GA and the DNA-GA

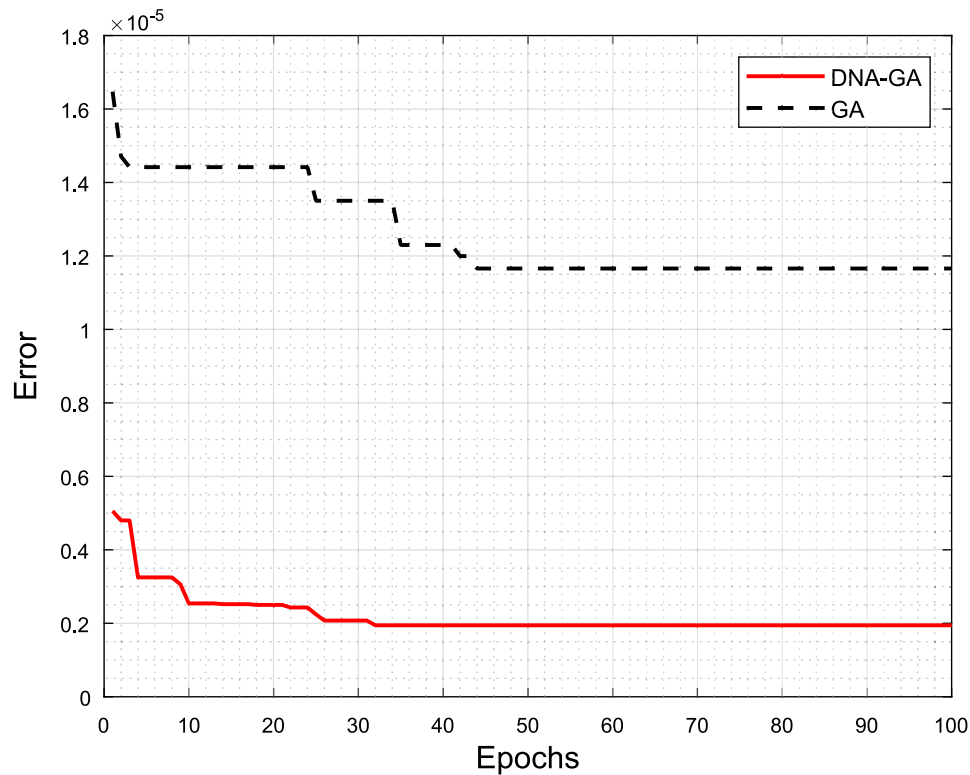


Table 4 Optimization of fitness function in iterative process

Fitness function initial value	Transcription factor	Post-transcriptional fitness value	Variation factor	Variation fitness value
46.51	0.12	47.62	0.09	48.78

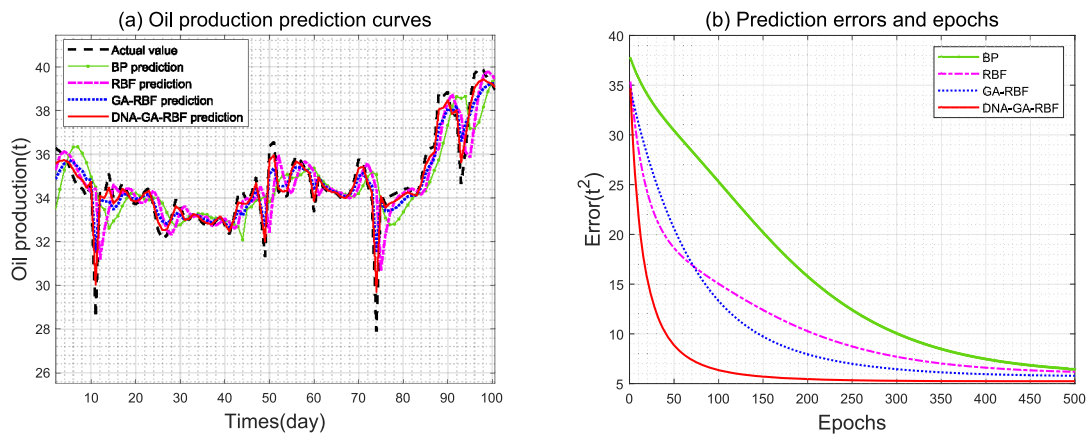


Fig. 13 Comparison of different models used in oil production prediction

As shown in Fig. 13, the traditional BP neural network models have slower convergence and less accuracy. Although the RBF neural network model can improve the convergence rate of the model, it does not significantly enhance the accuracy. The GA can enhance both the training speed and accuracy of the RBF neural network model, but its optimization efficiency decreases as the training error of the objective function decreases.

The DNA-GA not only improved the accuracy of the RBF neural network prediction model, but also converged faster. However, the efficiency of the algorithm during the early stages still needs improvement.

To ensure the robustness of the results, we conducted 30 comparative tests using various algorithms and obtained the average of the evaluation metrics from these 30 experiments. The results are presented in Table 5.

Table 5 compared the evaluation metrics of oil production models on the test dataset. Based on the first three evaluation metrics, the DNA-GA-RBF neural network model outperformed the GA-RBF model and the RBF neural network model. Compared to the RBF model, the DNA-GA-RBF model demonstrated a reduction of 74.2%, 7.34% and 80.3% in MAE, MRE and MSE of respectively. In comparison to the GA-RBF model, the MAE decreased by 50.3%, the MRE decreased by 4.95% and the MSE decreased by 69.6%. However, the running time of DNA-GA-RBF was obviously longer than other models. These results demonstrated that the DNA-GA had superior optimization power and the DNA-GA-RBF neural network model had higher training accuracy, but the optimization time needed to be improved.

5 Discussions

The proposed DNA-GA is a novel GA that builds upon improved genetic operators. In comparison to the traditional GA, the DNA-GA's genetic operator is modified according to changes in individual fitness, resulting in increased adaptive ability. The adaptive improvement of DNA-GA operator is embodied in adaptive transcription factor and adaptive mutation parameter. In traditional GA, crossover rate and mutation rate are set to specific values, which will reduce the diversity of population, lead to inbreeding, and increase the risk of premature convergence and falling into local optimum. In the DNA-GA algorithm, both are related to the fitness value of individuals, which changes in the iterative process. Formula (2) can dynamically adjust the crossover probability according to the fitness value of individuals, so that individuals with high fitness value have smaller crossover probability, thus accelerating the convergence speed of the algorithm; Formula (3) can also dynamically adjust the mutation probability according to the fitness value of individuals, so as to increase the diversity of the population and avoid the

algorithm falling into local optimization. In Fig. 10b, the optimization results of DNA-GA and GA on Rosenbrock show that DNA-GA has faster convergence speed and higher optimization accuracy, which can also be proved by the results recorded in Table 2. However, DNA-GA has several limitations: (1) the optimization performance is not consistently stable: even under the same maximum number of iterations, it may not perform as well as the traditional GA; (2) the genetic operator of DNA-GA is improved based on changes in population fitness, which makes the calculation method complex, and some parameters still require subjective tuning; (3) DNA-GA has low efficiency in the early stages of optimization, as evidenced by the experiments of test function optimization, which showed that DNA-GA's minimum iteration is not superior to that of the traditional algorithms; and (4) the DNA-GA algorithm has a long running time and slow optimization speed, which becomes more problematic when applied to complex models. To address these challenges, future research will explore several enhancements. We plan to apply different search approaches into distinct phases of the optimization process, such as block-level and network-level search to improve early-stage efficiency and reduce overall computational complexity [39]. Furthermore, we will integrate evolutionary strategies to enhance convergence speed and robustness [39]. Finally, we aim to incorporate advanced stochastic optimization techniques to enhance DNA-GA's ability to handle uncertainty in real-world applications [40].

Additionally, to enhance the generalization ability of the improved oil production prediction model based on DNA-GA, it is necessary to make the model simple and stable, reduce parameter redundancy, and account for the differences in characteristics between oil fields in different regions. Practically, the RBF oil production prediction model improved by DNA-GA requires suitable input features, such as water injection and pipeline pressure, which should be filtered and normalized before determining the population size, initial gene operators, and the number of optimization iterations.

Regarding the applicability of the model, we need to analyze the specific problems. Because of its excellent convergence speed and adaptive ability, our model is suitable for solving nonlinear complex system problems with high time series dependence and great influence by correlation and uncertainty, such as oil production prediction, and can obtain good prediction accuracy. Therefore, we can consider applying it to power load forecasting, stock price forecasting in financial field and river water flow forecasting. In addition, we can try to apply DNA-GA model to solving optimal solution problems, pattern recognition, data mining and image processing. For example, in machine learning, the training algorithm of artificial neural network can be improved into more advanced algorithms, such as gradient descent based on back propagation, improved gradient descent, momentum gradient descent, etc., so that it can fit the training data

Table 5 Comparison of evaluation metrics of oil production prediction models on the test dataset

Model	Performance metrics			
	MAE (%)	MRE (%)	MSE (%)	TIME(s)
BP	41.4727	17.3637	7.3631	3.4574
RBF	13.0145	12.0715	6.1323	3.7340
GA-RBF	6.5072	9.6818	1.8408	3.7480
DNA-GA-RBF	3.2536	4.7309	0.5602	14.8360

more accurately. In addition, the algorithm improvement can also be used to optimize various problems, such as depth first, breadth first, iterative deepening, etc., so that it can find the best solution faster and reduce the complexity of space and time.

6 Conclusion

We proposed a bio-inspired model for oil production prediction to address the challenge of low accuracy in oilfield development. This model utilized a new algorithm DNA-GA to optimize RBF oil well production prediction model. DNA-GA was designed based on the DNA regulation mechanism for selection, crossover, and mutation operators with self-adaptive improvement. The test function optimization results showed that DNA-GA had significantly improved convergence precision and speed compared to three existing optimization algorithms.

In addition, we successfully applied DNA-GA to optimizing the weight threshold of RBF neural network with gradient descent. Experimental results demonstrated that the RBF neural network optimized by DNA-GA was more effective in predicting oil production compared to the RBF neural network optimized by other algorithms. It showed significant improvements in both accuracy and convergence speed.

However, DNA-GA algorithm also has some defects, such as unstable optimization and low initial optimization efficiency, so some measures can be taken in the future work. Firstly, optimization efficiency can be improved by simplifying the calculation of genetic operators and reducing subjective tuning of parameters, such as using the immune mechanism of cell coordination to adjust parameters [41]. Secondly, the early-stage efficiency of DNA-GA can be optimized by implementing self-adaptive improvements in the early stages to reduce the impact of uncertain factors, such as random numbers. Thirdly, DNA-GA's application can be extended beyond updating the weight threshold of the RBF neural network to include hyperparameters optimization of more complex deep learning algorithms. Finally, optimizing the details of the algorithm, reducing unnecessary calculations and operations, terminating unnecessary loops and

iterations as early as possible, or using GPU to speed up complex models can improve performance.

Acknowledgements B. Liu and Z.R. Zhu were supported by The Fundamental Research Funds for the Central Universities, China (20CX05006A).

Author contributions Bao Liu: Methodology, Formal analysis, Resources, Writing—Reviewing and Editing. Yuqing Liang: Data Processing, Writing- Original draft, Draw Pictures. Zirun Zhu: Draw Pictures, Validation. Lei Gao: Conceptualization, Writing—Reviewing and Editing, Supervision.

Funding Open access funding provided by CSIRO Library Services.

Data availability No datasets were generated or analysed during the current study.

Declarations

Conflict of interest The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. Albertoni A, Lake LW (2003) Inferring interwell connectivity only from well-rate fluctuations in waterfloods. *SPE Reservoir Eval Eng* 6(01):6–16
2. Younk N, Hoffman BT (2022) Employing regression analysis to demonstrate the impact of hyperbolic decline curve parameters on long-term production forecast accuracy for unconventional oil and gas production in the bakken and barnett, *SPE Western Regional Meeting*, pp D021S011R006
3. Salazar M, Gonzalez H, Matringe S, Castiñeira D (2012) Combining decline-curve analysis and capacitance-resistance models to understand and predict the behavior of a mature naturally fractured carbonate reservoir under gas injection. In: *SPE Latin America and Caribbean Petroleum Engineering Conference*, pp. SPE-153252-MS

4. Cao F, Luo H, Lake LW (2015) Oil-rate forecast by inferring fractional-flow models from field data with koval method combined with the capacitance/resistance model. *SPE Reservoir Eval Eng* 18(04):534–553
5. Zhang B, HeHu ZY (2025) Neural network-based ground stress field inversion and fault influence study in south Sichuan shale gas reservoirs. *Carbonates Evaporites* 40(1):1–14
6. Liao Q, Xue L, Lei G, Xu L, Sun S, Patil S (2022) Statistical prediction of waterflooding performance by K-means clustering and empirical modeling. *Pet Sci* 19(03):1139–1152
7. Jahandoost A, Torghabeh AF, Hosseini AS (2024) Crude oil price forecasting using K-means clustering and LSTM model enhanced by dense-sparse-dense strategy. *J Big Data* 11(1):117–117
8. Ma K, Wu C, Huang Y (2024) Oil well productivity capacity prediction based on support vector machine optimized by improved whale algorithm. *J Pe Explor Prod Technol* 14(12):3251–3260
9. Li J, Bian X (2024) Improved least squares support vector machine model based on grey wolf optimizer algorithm for predicting CO₂-Crude Oil minimum miscibility pressure. *Energy Technol* 12(5):7–12
10. Liu B, Gao L, Li B, Marcos-Martinez R, Bryan B (2020) Nonparametric machine learning for mapping forest cover and exploring influential factors. *Landsc Ecol* 35:1683–1699
11. Ibrahim NM, Alharbi AA, Alzahrani TA, Abdulkarim AM, Alessa IA, Hameed AM, Albabtain AS, Alqahtani DA, Alsawwaf MK, Almuqhim AA (2022) Well performance classification and prediction: Deep learning and machine learning long term regression experiments on oil, gas, and water production. *Sensors (Basel)* 22(14):5326
12. Mohammadian E, Kheirollahi M, Liu B, Ostadhassan M, Sabet M (2022) A case study of petrophysical rock typing and permeability prediction using machine learning in a heterogeneous carbonate reservoir in Iran. *Sci Rep* 12(1):4505
13. Sharipov MI (2021) Development of a method for controlling the production process in oil and gas fields using neural networks. *Memetic Comput Adv Autom II*:3–14
14. Panja P, Jia W, McPherson B (2022) Prediction of well performance in SACROC field using stacked Long Short-Term Memory (LSTM) network. *Expert Syst Appl* 205:117670
15. Cheng X, Hu X, Li Z, Geng C, Liu J, Liu M, Zhu B, Li Q, Chen Q (2022) Using genetic algorithm and particle swarm optimization BP neural network algorithm to improve marine oil spill prediction. *Water Air Soil Pollut* 233(8):2022
16. Zhang Y, Wang R, Wang X (2021) Prediction method of wax deposition rate in crude oil pipeline based on RBF neural network and support vector machine. *E3S Web Conf* 271:04007
17. Ab Aziz MF, Mostafa SA, Mohd. Foozy CF, Mohammed MA, Elhoseny M, Abualkashik AZ (2021) Integrating elman recurrent neural network with particle swarm optimization algorithms for an improved hybrid training of multidisciplinary datasets. *Expert Syst Appl* 183:115441
18. Abdel B, Mohamed R, Hezam I, Sallam K, Hameed I (2024) An efficient binary hybrid equilibrium algorithm for binary optimization problems: analysis, validation, and case studies. *Int J Comput Intell Syst* 17(1):1–3
19. Alfarizi MG, Stanko M, Bikmukhametov T (2022) Well control optimization in waterflooding using genetic algorithm coupled with artificial neural networks. *Upstream Oil Gas Technol* 9:100071
20. Song L, Liu C, Shi H, Zhu J (2022) An improved immune genetic algorithm for solving the flexible job shop scheduling problem with batch processing. *Wirel Commun Mob Comput* 2022(1):2856056
21. Zhu S, Cai J, Sun E (2023) Mobile edge computing offloading scheme based on improved multi-objective immune cloning algorithm. *Wirel Netw* 29(4):1737–1750
22. Liu B, Xu M, Gao L, Yang J, Di X (2022) A hybrid approach for high-dimensional optimization: Combining particle swarm optimization with mechanisms in neuro-endocrine-immune systems. *Knowl-Based Syst* 253:109527
23. Liu B, Yang J, Gao L, Thiruvady D, Nazari A (2021) Bio-inspired heuristic dynamic programming for high-precision real-time flow control in a multi-tributary river system. *Knowl-Based Syst* 230:107381
24. Zang W, Ren L, Zhang W, Liu X (2018) A cloud model based DNA genetic algorithm for numerical optimization problems. *Futur Gener Comput Syst* 81:465–477
25. Jasra B, Hassan Moon A (2022) Color image encryption and authentication using dynamic DNA encoding and hyper chaotic system. *Expert Syst Appl* 206:117861
26. Nazin V, Poznyak S (2024) Non-quadratic proxy functions in mirror descent method applied to designing of robust controllers for nonlinear dynamic systems with uncertainty. *Comput Math Math Phys* 64(4):820–832
27. Li D, Pan Z, Hu G, Anderson G, He S (2021) Active module identification from multilayer weighted gene co-expression networks: A continuous optimization approach. *IEEE/ACM Trans Comput Biol Bioinf* 18(6):2239–2248
28. Franciskus A, Taviti N, Chamandeep K (2024) Advanced hybrid CNN-Bi-LSTM model augmented with GA and FFO for enhanced cyclone intensity forecasting. *Alex Eng J* 92:346–357
29. Xie F, Sun Q, Zhao Y, Du H (2022) An improved directed crossover genetic algorithm based on multilayer mutation. *J Control Sci Eng* 2022(1):4398952
30. Yin Y, Yu L, Jie S, Jia X (2023) Genetic algorithm based on grid maps for solving robot path planning problem. *Eng Lett* 31(4):180–189
31. Zhang S, Liang Y, Xu Z, Li H, Wang G, Cheng K (2024) (2024) Optimal scheduling of active distribution network based on RBF-stochastic response surface method. *Electric Power Syst Res* 232:110363
32. Zhang D, Wang H, Shan S, Wang C (2023) Dynamic surface adaptive RBF neural network control for a class of non-linear uncertain systems. *Electron Lett* 59(23):1–4
33. Guo Y, Gong J (2020) Group targets tracking using maximum entropy fuzzy based on fire-fly algorithm and particle filter In: 2020 7th International Forum on Electrical Engineering and Automation (IFEAA), pp 937–942
34. Hu Y, Zhang Z, Zhang Z, Li Z, Tang Q (2024) Matheuristic and learning-oriented multi-objective artificial bee colony algorithm for energy-aware flexible assembly job shop scheduling problem. *Eng Appl Artif Intell* 133:108634
35. Zhao J, Mark P, Vincent Y, Zhang Z (2024) A hybrid genetic search and dynamic programming-based split algorithm for the multi-trip time-dependent vehicle routing problem. *Eur J Oper Res* 317(3):921–935
36. Yue M, Guo B, Hu T, Guo X (2009) The research of parameters of genetic algorithm and comparison with particle swarm optimization and shuffled frog-leaping algorithm. In: 2009 2nd International Conference on Power Electronics and Intelligent Transportation System (PEITS), pp 77–80
37. Huang J, Zeng J, Bai Y, Cheng Z, Qi L, Liang D (2021) Layout optimization of fiber Bragg grating strain sensor network based on modified artificial fish swarm algorithm. *Opt Fiber Technol* 65:102583
38. Pil RS, Hazlie M, Ru WW, Azam MM, Nadzirah MN (2022) Optimal coordination of directional overcurrent relay based on combination of firefly algorithm and linear programming. *Ain Shams Eng J* 13(6):101777
39. Zhang H, Hao K, Gao L, Wei B, Tang X (2024) Enhanced gradient for differentiable architecture search. *IEEE Trans Neural Netw Learn Syst* 35(7):9606–9620

40. Zhang Z, Guo C, Wei Q, Guo Z, Gao L (2021) A bi-objective stochastic order planning problem in make-to-order multi-site textile manufacturing. *Comput Ind Eng* 158:107367
41. Liu B, Xu M, Gao L (2024) Enhanced swarm intelligence optimization: Inspired by cellular coordination in immune systems. *Knowl-Based Syst* 290:111557

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.